

1 Problem Statement, Overview, and Objectives

Differential testing [1] has long been considered one of the most powerful approaches to ensure the security, reliability, and correctness of software, *when applicable*. Differential testing offers a unique solution to the difficulty of specifying what a software system should (and should not) do: in essence, it tests a function f by checking its behavior matches function g that is *an (ideally independent) implementation of the same functionality*, and defines test failures as cases where $f(x) \neq g(x)$. The “functions” f and g in realistic settings are seldom actual code-level functions, of course, but more typically complex systems ranging from data-structure implementations to complex parsers, cryptographic protocols [2] and, especially, compilers [1] and file systems [3].

The ubiquity of differential approaches to testing compilers and file systems is no accident: these are two critical pieces of system infrastructure where genuinely independent implementations are often available, and standards such as ANSI C and POSIX make comparison reasonable, if not trivial; the same applies, though less frequently, to cryptographic algorithm and protocol implementations. Since file systems may rest on different hardware substrates, and compilers may make different choices where a language like C allows for implementation-defined behavior, in practice even in these best-case scenarios, the problem is usually not one of checking that $f(x) = g(x)$, but of checking that $O_f(f(I_f(x))) = O_g(g(I_g(x)))$, where O_f , O_g , I_f , and I_g are respectively, output and input abstraction functions for f and g , that allow for known differences in behavior and input spaces, and there are often also known inputs that violate the differential specification in a known and allowed way, making the full specification of the complex structure $P(x) \Rightarrow O_f(f(I_f(x))) = O_g(g(I_g(x)))$, where the guard P and O and I abstraction functions may also be quite complex.

Despite these difficulties, differential testing is extremely attractive, especially in settings where inputs are automatically (and stochastically) generated, such as random testing and *fuzzing*. Indeed, arguably the fundamental limitation of modern fuzzing, despite its huge success in finding software vulnerabilities and bugs, is the typical weakness of fuzzing oracles. It is extremely hard to determine if a system is behaving correctly on an arbitrary, often extremely exotic, input. This leads to results such as the difference between functional tests and fuzzing harnesses for Bitcoin core, where equivalent, almost 100% branch and code coverage yields huge discrepancies in mutation detection, with functional tests detecting almost 80% of injected bugs while fuzzing, despite perhaps more aggressive exploration of the state space, detects only 12% of these bugs [4]. A differential specification makes it possible for fuzzing to detect subtle semantic errors, not just memory safety violations and assertion failures.

It seems likely that the importance of differential fuzzing will only grow in the future, for two reasons, both deriving from the use of LLMs and agentic coding. First, the primary reason differential fuzzing is seldom used in practice is that the cost of an independent implementation of a software system, even in security or safety critical settings, is usually prohibitive. This cost is especially high given that previous work has shown that “independent” implementations are seldom as independent (in terms of correlation of bugs) as desirable [5]. This means that differential testing via separate implementations of functionality is usually only performed in cases of extremely complex specifications implemented by radically separated teams (e.g., compilers and file systems) with orthogonal incentives to produce their own implementations, or for very simple specifications such as data structures. Code synthesis by AI models sidesteps this problem: once a specification that produces a single workable implementation is available, having a single model produce multiple implementations is often relatively cheap. Using multiple models or changing prompts to focus code generation on, for example, a faster implementation, an implementation with a smaller memory footprint, a more readable implementation, an implementation using an unusual algorithm, and so forth, seems likely to produce controlled divergence that may exceed that of human developers. If the development cost of multiple implementations is radically reduced, the advantages of differential specification are clear.

Furthermore, one pre-AI way to work around the cost of a second implementation, of course, is to use differential fuzzing in a *regression* setting, comparing a new version of software to an older, better-tested

version. However, in practice, new versions of a software system are often introduced precisely because the functional interface has changed in some way, and the work required to conceive, implement, and maintain a mapping between new and old versions (which will be less stable, as each new version tends to remove some parts of the current mapping, as the “old” version is now the version that was new, as well as add new changes) discourages use in this setting, especially as the bug correlation problem reduces the power of differential testing. Again, LLMs are probably well-suited to constructing and maintaining version mappings at low cost¹. Therefore, differential testing (and fuzzing) is *much more practical and widely applicable in an era of agentic coding*.

Second, differential approaches seem ideal for probing ambiguities in prompts/specifications presented to AIs, and AI misunderstandings: if multiple implementations are produced from a specification, and checked for equivalence, supervisors of AI-driven development can examine cases where the AI’s “view” of the software’s behavior is unstable, and can work to remove ambiguity and correct mistakes. Agentic coders can automatically refer ambiguities they cannot resolve, discovered during their own automatic testing of multiple efforts to satisfy a specification, to human judges. Therefore, differential testing (and fuzzing) is *also a potentially critical technology for ensuring AI-generated code actually satisfies fully specified human goals*.

Differential fuzzing is, of course, already an important practice, particularly in security-critical domains and core systems infrastructure development. However, differential fuzzing as practiced now is often limited in effectiveness. All too often, the approach taken is to independently fuzz two implementations, then compare their behavior over the union of the produced corpuses of interesting inputs. While this can discover bugs, it is effectively throwing away most of the power of fuzzing, in two important ways. First, the implementations are only tested differentially for a small number of inputs, the “final outputs” of fuzzing runs on the two separate systems; this misses out on the opportunity to detect divergence and bugs on potentially millions or billions of executions. Second, the two independent corpuses are only derived from efforts to cover behaviors of the two systems independently; if an input does not produce novelty in terms of either system in isolation, but represents a unique *relationship between coverage patterns* for a shared input, this method is simply incapable of identifying and exploiting such inputs. Such inputs, however, are, intuitively, the most promising in terms of detecting divergences in implementation behaviors.

With somewhat more effort, users of differential fuzzing can compile two implementations in such a way that a fuzzer can see the combined coverage of both systems, essentially a program path over the merged program `r1 = f(x); r2 = g(x); assert (r1==r2);`. This overcomes the above objections in part, but still in fact leaves much to be desired, in terms of taking advantage of the differential nature of the specification.

In computer science, fully leveraging an approach tends to require making it truly *first-class*, supported by language and analysis tools; without OO constructs or language-native threads, objects and concurrency are limited in applicability, and the recent jump in sophisticated usage of functional programming techniques in mainstream languages can essentially be traced to the widespread availability of real lambdas and closures, rather than weaker proxies such as object dispatch simulations of lambdas, or C’s function pointers.

Similarly, differential specification (arguably the most powerful and “usable” of all specification techniques) can only be fully exploited when *fuzzing algorithms* and *specification techniques* are differential-aware. This requires abstracting away from the specifics of a particular instance of differential testing/fuzzing. The most important previous work along this line, the NEZHA framework of Petsios et al. [6] offered a start on such an approach, and proved extremely successful (finding differences between implementations at a rate 20-50 times higher than previous approaches) in practice, but failed to truly offer up first-class differential fuzzing.

This proposal aims to develop practical first-class differential-aware fuzzing algorithms and highly effective automated and manual methods for building differential specifications, that is constructing functions of the form $P(x) \Rightarrow O_f(f(I_f(x))) = O_g(g(I_g(x)))$. The underlying logic of this proposal is that differential testing’s potential is, at present, greatly limited by the lack of such first-class tools for exploiting the fact that a very

¹We believe this will be especially true if LLMs have access to a high-quality DSL for constructing differential harnesses, which is one of the goals of this proposal.

large number of software problems can be defined as doing “(almost) the same thing” as another system. This proposal aims to address that limitation via *first class differential fuzzing* and *first class differential specification*; in particular the first idea introduces completely new notions of *relational coverage* and *provenance-based* interestingness of observed program behaviors.

2 Intellectual Merit

The essence of this proposal is the hypothesis that software security and reliability could be greatly enhanced by development of concepts, algorithms, and tools for exploiting the similarity of many software systems. The case of comparison of different versions of the same system is the most obvious of these, where differential testing is a kind of abstraction of regression testing; the most important case for software security is the large body of fundamental systems software that implements functionality that is repeatedly implemented because it is so foundational to a variety of software systems. Such “ubiquitous” software artifacts (e.g., compilers, file systems, cryptographic algorithms, regular expression tools, data structures) are notably also frequently involved in particularly dangerous software vulnerabilities, because they are so widely used in different systems, and invoked in low-level code including core security functionality and operating system contexts that operate with elevated privileges or direct impact on security properties of a system.

2.1 First-Class Differential Fuzzing

In order to understand the limits of current approaches, it is useful to think about a very simplified description of what a modern mutation-based fuzzer such as AFL++ [7], libFuzzer [8], or Google’s Centipede [9], does, at a high level that essentially applies to all widely-used fuzzers:

1. First, a fuzzer selects an input, a *seed* that it has already executed (possibly presented to it as part of an initial *seed corpus* of interesting inputs, e.g., valid media files when fuzzing a media parser), based on a *seed scheduling* algorithm that selects the most promising such inputs to work with. At this stage the fuzzer may also compute an *energy*: how much time to spend working on this particular seed (energy can also be modified once the fuzzer starts seeing results of working with the seed, but we are offering a simplified version of the process here).
2. Then, repeatedly, up until the limits set by *energy*, the fuzzer *mutates* the seed, making a randomly chosen change to the input; these changes can be simple byte-based mutations (often referred to as “havoc” operators) or more targeted and domain-specific, as in some compiler-fuzzing tools [10].
3. For each such modified variant of the seed, the fuzzer executes the mutant of the seed; if a crash is detected (in differential fuzzing, the executable should be set up so that divergence in behaviors triggers a crash) the input is preserved as representing a potential bug. If the input does not cause a crash, the fuzzer determines if the input triggers *novel* behavior. The definition of input novelty can vary; some fuzzers only consider inputs that cover new branches or basic code blocks novel (this is libFuzzer’s default mode, for example); other fuzzers (AFL and family, for example) make use of a more complex edge-hit-count pattern approach that can approximate a form of (acyclic) path coverage [11]. If the input is novel, it is added to the set of seeds to schedule and spend energy on collected by the fuzzer, and the loop eventually returns to step 1, when energy for the current seed is exhausted. The identification of novel inputs is what lets the fuzzer slowly accumulate new interesting program behaviors and deeply explore the program, searching for bugs.

NEZHA’s fundamental contribution was making a tuple representation of coverage across compared methods, $\langle \text{cov}(f), \text{cov}(g) \rangle$. This is a huge advance on simply attempting to cover both implementations independently, then compare over interesting paths for either approach; it captures the importance of asymmetry and unusual *pairings* of coverage paths between compared implementations. However, this is in essence already done by AFL/AFL++ and other fuzzers, if the differential targets are compiled into a single instrumented binary, and NEZHA’s experimental evaluation unfortunately only compares to application of AFL to generate inputs based on a single implementation. NEZHA’s δ -based measure of input novelty is essentially based on a flattening of output (in black-box settings) or program path (in grey-box settings)

tuples, so that novel pairings of f and g behaviors are observable; when a single executable with a global path is produced from the differential fuzzing targets in AFL/AFL++, the same result is essentially achieved. The advantage NEZHA shows over an implementation of what they call “global coverage” in libFuzzer is due to libFuzzer’s simpler novel branch/block heuristic for determining which inputs are interesting.

However, despite missing the fact that their grey-box approach to δ -based novelty is likely simply equivalent to a single-compiled-binary AFL/AFL++ approach, the NEZHA paper shows how to *take differential fuzzing seriously*. This proposal aims to go beyond that initial approach by modifying fuzzer internals (implemented on top of the industry-standard AFL++ platform) to take advantage of the differential problem in all stages of the basic fuzzing algorithm presented above:

- First, and most important, we aim to extend the notion of novelty to preserve, and thus explore from, inputs that do not find novel behavior of either f or g alone, but which (going beyond NEZHA and AFL-family methods) expose cases where either of f or g *changes behavior* more in response to a shared change in an input seed. Intuitively, cases where one implementation’s behavior is much more affected by a change to an input than the other are “on the path” to exposing full behavioral divergence. This requires a new approach to novelty evaluation not previously explored in the fuzzing literature to our knowledge (relative/response-based novelty), where the behavior of a mutated seed is considered *relative to the original seed behavior*.
- Second, the extent of such changes can be used to inform scheduling and energy decisions; the more input changes differently affect behavior, the more the input merits urgent investigation and substantial computation resources.
- Third, these basic ideas can be further extended in the case where f and g share a common code base, since f is a new version of g , not an independent implementation. In these cases program edges can be mapped into sets of shared and differing code locations, and the structure of the code changes can be exploited to better focus on the changes made to g .
- Finally, the fuzzer can identify when certain byte locations (or higher-order structure entities, in theory) or mutation operators are especially prone to produce large divergences in response for f and g and spend more time applying those more-productive mutants.

Fundamentally, these ideas can be summarized as taking the fact that differential fuzzing differs from most fuzzing in that it has a mapping between two pairs of behaviors to exploit seriously. In order to make this concept clear, it is easiest to focus on the first proposed algorithmic change, which seems most novel and most promising.

Consider two functions f and g that implement a critical cryptographic function, where the g is a more complex, but more efficient, implementation of the same algorithm. The specification here is simple: the two functions should, given the same inputs (e.g., keys and ciphertext, or protocol message), yield the same outputs (plaintext, protocol configuration structure). Now, consider the behavior of a traditional AFL-style harness calling both functions on a set of inputs A, B, C, D, and E. Suppose the harness is constructed such that first f is called, then g , and the paths taken by the functions, where f_1 etc. are path edges traversed in the respective functions are:

Input	f path	g path
A	f_1, f_4, f_6	g_1, g_3, g_4, g_5
B	f_1, f_4, f_6	g_1, g_2, g_4, g_5
C	f_1, f_2, f_3	g_1, g_3, g_4, g_5
D	f_1, f_2, f_3	g_1, g_3, g_4, g_6
E	f_1, f_4, f_6	g_3, g_2, g_8, g_9

All of these inputs will be saved as novel, since they all execute an overall new sequence of edges. This is the case despite the fact that input C, for example, does not produce novel behavior in either f or g alone, it is only novel with respect to both functions. That such sequences need to be seen as novel for effective differential fuzzing is a key insight of NEZHA.

Now consider the input F. F produces the path $f_1, f_4, f_6, g_1, g_3, g_4, g_5$. Should F be saved as an important

input to explore from? AFL++ or the NEZHA approach would say no; F produces the same path through f and g as A, the very first input explored. However, this ignores how F was derived: F is the result of mutating input E. The small change made to the input did not change the behavior of function f but dramatically changed the behavior of function g . Had F been produced from D, on the other hand, the behavior of *both* f and g would be seen to be changed, and the previously seen path therefore uninteresting. This potentially highly significant fact is invisible unless the fuzzer is aware of the structure of the fuzzing problem, and both distinguishes f and g behaviors *and* makes use of the *provenance* of fuzzing inputs, so that behavior change or lack of change can be *observed* and *exploited* as a signal of behavioral divergence. In short, we aim to exploit a new kind of signal for fuzzing: *relational coverage* between compared programs, where not only coverage itself matters, but the *relation between changes in coverage, in terms of both existence and magnitude*, given fixed changes in input.

All of the above proposed differential adaptations of the core mutation-driven fuzzing algorithm are derived from this insight that once a fuzzing problem is seen as existing between two or more targets, this new *relational* notion of coverage becomes potentially very valuable. The exact nature of relational coverage concepts can be quite subtle, involving both static and dynamic notions of change.

For instance, the insight behind the third method listed above, which tries to exploit cases where f and g share a largely identical code base, is that when a change to an input produces changes in the *shared* code that does not differ between f and g this is likely to be highly significant: behavior has flowed out of the non-shared portions of the code base to influence the behavior in ostensibly identical parts of the implementation; the seemingly fixed and uninteresting relation between unchanged code regions is in fact non-fixed and complex, mediated by data flow from changed regions. Changes *wholly contained* in divergent parts of the code may be due to accidental aspects of the way the code behaves; this seems, intuitively, less likely when there is data flow of difference to shared code. This idea therefore makes use of relations between the differentially compared programs at both the static and dynamic level. If we simplify the model by assuming the changed code is contained in a single block, modifying a unit of functionality that is internally modified but intended to have the same black box behavior, we have identified a case where the two black boxes must be behaving differently, since code outside the box has changed behavior. The conclusion is less confident with more complex non-contiguous relationships between changed versions, but the basic argument for elevating the signal on divergence in shared code seems at minimum deserving of empirical exploration.

2.2 First-Class Differential Specification

Lack of fuzzer awareness of differential structure contributing to weakness in finding differentially-defined bugs is almost certainly *not* the primary reason differential testing is not more widely used, even ignoring the cost of developing independent implementations (which may now be mitigated by AI code synthesis). Many developers do not use differential testing methods because in practice, the problem is seldom one of comparing two implementations with perfectly identical behavior, and the effort required to make differential testing actually useful can seem overwhelming. While the first prong of this proposal will, we believe, greatly improve the effectiveness of differential testing for those already using it, it is unlikely to be widely used without accompanying advances in *differential specification*.

Gulzar et al. in 2019 reported on the (large and growing, at that time) adoption of differential testing at Google [12]. Their results, based on a survey of 117 software engineers, in-depth interviews with 5 experienced users of differential testing, and analysis of over 2,000 differential tests, showed two key problems with differential testing. First, differential tests, as practiced, usually based on data drawn from end-to-end execution of production systems, were noted as slow. This problem is in part addressable by better differential fuzzing, which can produce fast-executing tests that concentrate on difference-exposing tests. The second most common problem was that manual comparison of results is tedious and difficult, and clear pass/fail results are hard to obtain. This problem is therefore significant in the setting of the study, manual difference testing between different versions of the same software system. It is much more important in a differential fuzzing context, where hundreds or thousands of false positives may be generated, behavior is more likely to be found in obscure, ill-understood parts of the system state space, and the comparison may

be between truly independent implementations of a standard functionality, rather than versions of the same software system.

Differential testing harnesses such as that for the widely used Pyfakefs Python mock file system [13, 14] often devote up to half their code to allowance for divergences between a tested and reference system; the POSIX standard is, like many other important standards, partly nondeterministic. The intellectual effort required to construct such abstraction code and determine which divergences are important and which are spurious is a major burden on would-be users of differential testing, even when analysis reveals the categorization to be, in some sense, “obvious.”

Users of differential fuzzing can be divided, roughly, into two classes:

- Users expecting a high degree of automation, who do not want to invest a large human effort in constructing a differential specification. In such cases, often but not always focusing on different versions of the same software system, users are focused on ease-of-use of differential testing, not on its raw power. These users may be most likely to be developers of the code under test, and aim to use differential testing in large part to detect impact of code changes they have made outside the scope of their own unit tests and module [12].
- Users who are willing to devote considerable effort to producing a differential specification. These cases tend to be focused on using differential testing as a means for detecting very subtle bugs and obscure behaviors, not using differential testing primarily as an additional, easy-to-apply, extension of unit tests. File system, compiler, and cryptographic algorithm differential testing and other high-stakes systems software where non-developer testing experts may be involved is a paradigmatic instance of this kind of high-effort differential testing.

These two modes of differential testing/fuzzing naturally suggest two modes of specification. In the first case, the ideal implementation of differential fuzzing takes as input exactly two binaries, and compares their behavior, with no additional work required by the user. In some cases, “difference” may even be left as the simple case where one system crashes or returns an error code and the other does not.

In a few cases, this is sufficient. More commonly, however, considerable complexity remains: two systems may produce output on a channel such as a file or stdout/stderr that is almost identical, but diverges in, e.g., the content of text messages. In other cases, behavior may not be so simply observable, and users may want to compare less visible aspects of behavior, such as system call sequences, maximum memory footprint, or execution time. Of course, an AFL++ extension can be developed that allows users to specify such behavioral comparison standards on the command line, including potentially inline regexps or configuration files to control allowed divergence in text-based outputs, but the triage problem of understanding divergences is a major burden on users. We propose to not only implement and classify an extensive set of composable, customizable, behavioral comparison functions, but develop *automated calibration methods* that use the ability of fuzzing to produce a statistically large number of behaviors in order to estimate a good comparison function. Inferring a change in output message from a large body of stdout traces that differ only by a regexp substitution is the kind of task that can and should be automated, and a configuration generator can reduce the risk of such an approach by presenting a summary of consistent differences to users in an easily-understood format, e.g.:

- Version B consistently replaces ‘‘FILE OR DIRECTORY’’ with ‘‘FILE/DIRECTORY’’ in stderr messages
- Version B consistently reorders the `rename` and `close` POSIX calls from `rename --> close` to `close --> rename`
- Version B omitted a call to `clock_gettime()` from 95% of all observed executions; a set of representatives of the 5% of cases still calling this function were saved to the `exceptions/` directory

The second case is where the user of differential fuzzing aims to put in the effort to construct a harness that controls inputs to avoid areas of known divergence, to introduce complex behavioral pairings (e.g., where fault injection is performed in a simulated flash file system, and this has to be handled by intercepting and

```

pool: <int> 4
pool: <avl> 3
property: <avl>.check_balanced()
<int> := <[1..20]>
<avl> := avl.AVLTree()
<avl>.insert(<int>)
<avl>.delete(<int>)
<avl>.find(<int>)
<avl>.inorder()
~<avl> == ~<avl>
reference: avl.AVLTree ==> set
reference: insert ==> add
reference: delete ==> discard
reference: find ==> __contains__
reference: METHOD(inorder) ==> CALL(items)
compare: find
compare: inorder

```

Figure 1: A TSTL differential harness.

modifying calls to a “real” file system appropriately [3]), or where the mapping between inputs and outputs to the two systems is far too complex to infer from statistical observation, and must be modeled by a domain expert. Moreover, in such settings, there may be a goal of using the differential harness for multiple pairs of systems, with some kind of configuration file controlling the details of the particular specification to be used. In general, this problem is complex and unique enough to merit the use of a Domain Specific Language (DSL) [15, 16].

In previous work, we developed the TSTL (Template Scripting Testing Language) [17, 18], which was used for the testing of the Pyfakefs mock file system mentioned above [13, 14]. TSTL produces unit tests of Python programs using a nondeterministic, pool-based, approach, and includes unique features to support more complex forms of differential testing. A simple differential harness for an AVL tree implementation is shown in Figure 1. TSTL lets users specify a testing problem (here, that of testing an AVL tree) in terms of a single system under test (SUT) and provide mappings between test actions for the SUT and a reference implementation in the form of transforms that generate “mirror” actions to be performed on the reference. Here, for example, a new `AVLTree` construction is always mirrored by creation of an empty Python set. More complex transforms than mere name changes are also possible, e.g., where `inorder` traversal of an AVL’s contents (a method call) is mapped to a function call on a set. Users can specify when the return values of actions are to be compared for equivalence, and can provide Python-coded abstraction functions over inputs or outputs as needed. The TSTL approach was designed to enable differential testing in settings such as file systems where system behavior over a series of stateful actions is to be compared, not just single function calls.

While the TSTL approach is effective, allowing the complex Pyfakefs harness, which resulted in discovery and correction of more than 100 bugs in Pyfakefs (and one bug on Mac OS/BSD `rename`), to consist of only 124 non-comment lines, while supporting full reference comparison of both Windows and Mac OS/linux filesystems with Pyfakefs mocks, it also has serious limitations.

Most importantly, any DSL for differential testing is likely to either embed a full programming language (as does TSTL) or be embedded in a full programming language [16]. The TSTL approach has benefits, including the elimination of redundant and complex code for representing nondeterministic choice of test actions, but in general for most differential fuzzing problems it is preferable to embed the DSL in a host language. TSTL’s adoption was always limited by the need to learn a new programming language, a potentially daunting

task; testing and verification systems such as JUnit [19], GoogleTest [20], CBMC [21], FuzzTest [22], and DeepState [23] that “meet developers where they are” [24] without requiring users to learn a new language have always proved more successful and widely adopted. In particular, most of the critical target programs for complex differential harnesses are written in C or C++ (like most systems and low-level software), and ideally a solution would have the following features:

1. Easily support testing of C and C++ target code.
2. Integrate seamlessly with existing tools for expressing property-driven fuzzing problems in C and C++, such as Google FuzzTest [22] and Trail of Bits’ DeepState [23, 25].

FuzzTest is a promising system to integrate with because it is widely used and has the institutional credibility of Google behind it; it also provides a good baseline for non-first-class but known-effective fuzzing for differential problems. DeepState, in contrast, both provides full support for API-call-sequence fuzzing problems like file system testing (which is unfortunately inherently awkward using FuzzTest’s API) and can make use of our improved, differential-aware, versions of AFL++.

The critical difficulty is how to satisfy these requirements, which essentially require a DSL embedded within an existing language (for integration with FuzzTest/DeepState), without losing the power of TSTL’s notation to express “mirroring” of actions by transforms of calls or actions on a single target system, which is extremely concise and expressive, and removes many potential sources of error in writing a differential harness. In practice, determining the best way to do this will require substantial effort, but our preliminary experiments in this direction suggests that a kind of hybrid DSL approach, with a comment-embedded language (like doxygen, JML, and other methods that work via semantically-meaningful comments) that constructs macro definitions that safely apply transforms to seemingly single-target C and C++ code is one possible solution. This would mix the typical FuzzTest/DeepState approach to expressing calls to tested functions, where such calls simply appear as normal C/C++ code, with an auto-coding step where a set of TSTL-style reference and other transform indicators map out an expanded version of the harness to compile and run. Such mappings could also potentially provide some support for call-sequences testing for FuzzTest, since a transform step is already being applied to the fuzzing harness.

Conceptually, the key feature of differential specification is that the core of the problem is *avoiding false positives*: in traditional, assertion-based testing, or even property-based testing, specification effort is usually largely invested in defining failure; in differential testing, the effort is in avoiding reporting spurious failures, since the default equality relation defines all but a very small set of executions as failing, with no additional effort. This means that in automatic calibration of differential specifications, the impetus is on the fuzzing to explore the system sufficiently to identify likely-spurious, systematic, differences and present them for user approval, and in manual harness construction the impetus is on the DSL to support ways to express qualifications and modifications of the differential specification that are frequently used, and invoke abstractions and type transforms in an easily read and easily modified way.

2.3 PI Qualifications

PI Groce has a long history with software analysis and verification, including involvement in design and development of well-known fuzzing and testing tools, including the TSTL testing tool for Python, and the DeepState unit-fuzzing interface for C++ and C code; he is at present lead developer and maintainer for multiple open source testing and fuzzing tools, with over 100 GitHub stars (and, in one case, more than 800 stars: DeepState); he has, in the past, managed releases of industrial open source testing tools with more than 3000 GitHub stars. Groce typically reports 50-100 bugs detected using such tools to open source projects each year, and has been awarded significant bug bounties for security-critical compiler bugs detected using his work. Groce was twice employed by the Bitcoin Core team to investigate the effectiveness of their fuzzing framework, a key component in ensuring the security of the leading implementation of the most important cryptocurrency; this effort included evaluations of the Bitcoin Core fuzzing team’s own internal work on differential fuzzing of Bitcoin versions. His work has consistently involved a marriage of theory of testing and analysis with practical work oriented towards real-world developers, implemented in working tools (all, as will be the case with this project, released as open source). Groce is author of one of the most-cited

papers on differential testing [3] (the 7th most cited paper on the topic, according to Google scholar) and the TSTL testing DSL for Python was the first (and, to our knowledge, thus far *only*) such language to include automation of differential harness construction, including complex mappings on inputs and outputs, as a first-class construct in declarative testing [26, 27].

3 Research Plan

3.1 Thrusts

There are two major thrusts of this proposal, both including conceptual and engineering aspects, with feedback between concept, evaluation, and engineering guiding the research activity.

The first thrust, and the most important, is the exploration of the ability of relational coverage concepts to improve the effectiveness of differential fuzzing, given a fixed, well-defined, problem of detecting behavior divergences among two or more implementations of the same functionality. We believe the core idea of relational coverage is likely to significantly enhance the effectiveness of differential fuzzing for bug detection. This aspect of the project will consist of implementing the simplest plausible versions of the kinds of ideas proposed above, and exploring their effectiveness via preliminary evaluations of the techniques using the same evaluation techniques as proposed below for the publication of the research. We expect some feedback from experiments to be necessary, since adding new core techniques to AFL++ will involve many detailed decisions that may impact fuzzing effectiveness but will either be underdetermined by the basic concepts or will involve fuzzer throughput driven tradeoffs (approximations similar to those in the standard fuzzer compression of coverage information, for example).

The second thrust involves two sub-parts: first, since many potential users of differential fuzzing are dissuaded from applying the technique by the cost of constructing a differential harness, we aim to develop a principled approach to automatic calibration of harnesses using a variety of possible difference signals (output to stdout/stderr, file outputs, time and space performance, etc.). Second, because some of the most interesting and important differential fuzzing problems require careful thought and complex abstraction mechanisms, or removing parts of the input space not handled by one or more differentially compared implementations, we will investigate the development of a DSL for differential harnesses, based on C++, that can integrate with the DeepSpace fuzzer and Google’s FuzzTest infrastructure, without depending on either. This aspect of the research will necessarily be more exploratory, initially involving efforts to transform differential harnesses from the literature into standalone implementations or potentially automatable AFL++ modes.

3.2 Products

Primary products of this proposal are therefore:

1. An implementation of differential-aware fuzzing algorithms as a new mode of operation for the AFL++ fuzzer, including novel seed scheduling, energy, and mutation approaches based on methods; these will include methods that generalized to all differential fuzzing problems and to the specialized case of versions with largely overlapping implementations.
2. Modalities for automating common cases where the task is differential testing of two binaries that operate over a file or standard input, the common use of AFL and other fuzzers: these will include calibration tools for taking binaries and automatically proposing a differential comparison scheme (composed of those available to users) that is sufficiently sensitive to detect subtle bugs, but applies simple abstractions and choices of detection channels to reduce obvious false positives.
3. A framework/light DSL for declarative, legible, differential harness construction that works with the above fuzzing algorithms and cooperates with the DeepState unit-fuzzing framework, but does not require use of DeepState.

3.3 Evaluation

These products will be evaluated in multiple ways. First, algorithmic fuzzing changes can be evaluated using field-standard methods for evaluating fuzzing research [28]. The same differential fuzzing problems

can be executed using both baseline AFL++ and combinations of proposed techniques (in the standard ablation study approach) to determine the overall effectiveness of proposed methods and of each proposed component. Benchmarks for this aspect of evaluation can be drawn both from existing differential harnesses available in open source repositories and in the differential fuzzing literature: for standard benchmarks in fuzzing literature, differential problems can be constructed by using different software versions with known divergences or by using source or binary mutation, removing all mutants detected by a standard corpus (forcing non-trivial differential problems). For problems taken from the differential fuzzing literature, including special-purpose differential fuzzing for a fixed domain, numerous real-world examples should be available, with complex problems across very different implementations, not just differencing versions of the same problem.

For these experiments studying the basic effectiveness of relational coverage and relationally-guided mutation in fuzzing, the most interesting benchmarks will correspond to standard fuzzing metrics: time to first detected divergence and number of known divergences discovered (we expect there may be tradeoffs between these, which we aim to explore in a more systematic way than most fuzzing evaluations).

The thrust of differential specification work focusing on binary pairs and automatic oracle generation can be evaluated by reporting false positive and false negative rates, determining the effectiveness of the auto-calibration modes.

Evaluating a manual harness construction DSL is less straightforward and quantitative; we plan first to simply compare code sizes of harnesses, based on problems from the differential fuzzing literature, implementing the same specification with and without the DSL constructs, which should demonstrate the ability of the approach to concisely express standard differential testing problems. A second approach, tied to our expectation that differential testing may be an important tool in AI-based code synthesis, is to pose problems in both harness construction and harness understanding to state of the art coding AIs, with and without the DSL. Our hypothesis is that given a good DSL and documentation for it, AIs will be able to construct more effective harnesses and answer questions about harnesses better than when differential specifications must be expressed in less concise code, without idiomatic “shortcuts.”

4 Related Work

Previous work on differential testing is considerable, but previous work on *general mechanisms* for either finding differences in versions or specifying differential comparison problems is relatively limited.

Differential testing (and fuzzing) for specialized settings: McKeeman’s paper on using differential testing to find bugs in compilers is the earliest and most influential work in the field [1]. Compiler testing (as well as other language tool testing, e.g., for debuggers [29] or static analysis tools [30]) has a long and distinguished line of work based in differential approaches [31, 32, 33]; indeed, by and large differential testing is the only means for finding wrong-compile bugs that is in widespread use, and is essential to the success of tools such as Csmith [34]. File systems have also been a target for differential approaches [3]. Recently machine learning systems have also been a focus of differential approaches [35, 36]. Security and privacy testing also makes significant use of differential approaches [37, 38]; a notable recent example is the work of Pengbo et al. [39] that discovered 11 novel root causes of X.509 certificate validation divergences between SSL/TLS implementations, and demonstrated that RFC 5280 is fundamentally ambiguous.

Differential testing and fuzzing as a first-class problem: As noted, the most important such work, with respect to detection of differential bugs, is probably that of Petsios et al. on the NEZHA framework [6]. As discussed extensively above, this work revitalized interest in the problem of working out *general purpose* methods of finding divergences between software systems using state-of-the-art fuzzers. While we hypothesize that some algorithmic fuzzing advances presented in the NEZHA work may be highly significant only for libFuzzer-like novelty functions, the general framework for posing the problem and focusing on executions that can reveal differences is a major precursor to this proposal’s approach.

PI Groce’s previous work in the TSTL testing framework for Python is, to our knowledge, the most extensive previous effort to make differential specification an easier problem, by mapping sequences of

actions over two implementations into each other, and allowing for use of abstraction functions over inputs and outputs in such a setting [26, 27].

5 Broader Impacts

Improving Software System Reliability: A key element of this proposal’s approach is to focus on realistically deployable techniques. All methods developed will aim for seamless integration with the AFL++ fuzzing system, and thus immediate practical applicability to real-world problems. Differential testing is widely used in various aspects of critical systems infrastructure, and improvements in the ability to detect bugs using differential methods are likely to contribute to software reliability. Additionally, as noted in the problem statement, it seems likely that differential approaches are both made more practical by, and essential for the reliability of, LLM and agentic coding efforts. It is likely that even future workflows with few humans in the loop will make use of differential techniques, and given that fuzzing driven by random mutations is likely an orthogonal source of bugs to LLM-based static analysis of code, such methods will be more important for finding bugs made by LLMs that they cannot reliably detect by only their own “native” methods.

Education and Outreach: The proposed research yields several opportunities for enhancing CS education, recruiting new CS majors, and retaining CS students. PI Groce will work with the NAU Student ACM Chapter to present a series of “excursions in testing” that introduce automated testing to students, using differential approaches to find bugs in real world code, including code from media player libraries. The work of Guzdial [40] has shown that media computation is an effective way to both recruit and retain students in computer science. Groce also teaches classes (undergraduate and graduate, including a required course in NAU’s Software Engineering major) on automated testing that can present project results. Groce’s prior engagement in teaching and mentoring high school students (including as summer students who have published their work in major venues) and working with local technically-oriented 4H chapters, including in rural communities (such as Williams, AZ) surrounding Flagstaff will also allow preparation and presentation of versions of these excursions tailored to high school students’ level of knowledge.

6 Results From Prior NSF Support

The most relevant prior NSF support for PI Groce is CCF-2129446, “Feedback-Driven Mutation Testing for Any Language,” with a total budget of \$500,000 from 9/2021 until 8/2024, a collaborative proposal with Claire Le Goues of Carnegie Mellon University. *Intellectual Merit:* This project focuses on allowing developers to improve testing by using mutation testing to identify weaknesses in tests. This project has resulted in six publications [10, 4, 41, 42, 43, 44] thus far and two major software releases. *Broader Impact:* Work from this project has already resulted in the reporting and correction of multiple bugs in software systems, including widely used production compilers. The focus on practical discovery of software bugs and compiler testing is a direct path to the proposed project’s approach.

References

- [1] William McKeeman. Differential testing for software. *Digital Technical Journal of Digital Equipment Corporation*, 10(1):100–107, 1998.
- [2] Jean-Philippe Aumasson and Yolan Romain. Automated testing of crypto software using differential fuzzing. *Black Hat USA*, 7:2017, 2017.
- [3] Alex Groce, Gerard Holzmann, and Rajeev Joshi. Randomized differential testing as a prelude to formal verification. In *International Conference on Software Engineering*, pages 621–631, 2007.
- [4] Alex Groce, Kush Jain, , Rijnard van Tonder, Goutamkumar Tulajappa Kalburgi, and Claire Le Goues. Looking for lacunae in bitcoin core’s fuzzing efforts. In *ACM/IEEE International Conference on Software Engineering*, pages 185–186, 2022.
- [5] J.C. Knight and N.G. Leveson. Correlated failures in multi-version software1. *IFAC Proceedings Volumes*, 18(12):159–165, 1985. 4th IFAC Workshop on Safety of Computer Control Systems 1985 (SAFECOMP’85): Achieving Safe Real Time Computer Systems, Como, Italy, 1-3 October 1985.
- [6] Theofilos Petsios, Adrian Tang, Salvatore Stolfo, Angelos D Keromytis, and Suman Jana. NEZHA: Efficient domain-independent differential testing. In *2017 IEEE Symposium on security and privacy (SP)*, pages 615–632. IEEE, 2017.
- [7] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. Afl++ : Combining incremental steps of fuzzing research. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. USENIX Association, August 2020.
- [8] Kosta Serebryany. Continuous fuzzing with libfuzzer and addresssanitizer. In *Cybersecurity, Development (SecDev)*, IEEE, pages 157–157. IEEE, 2016.
- [9] Google. Centipede: a distributed fuzzing engine. <https://github.com/google/fuzztest/tree/main/centipede>.
- [10] Alex Groce, Rijnard van Tonder, Goutamkumar Tulajappa Kalburgi, and Claire Le Goues. Making no-fuss compiler fuzzing effective. In *ACM SIGPLAN International Conference on Compiler Construction*, pages 194–204, 2022.
- [11] Milos Gligoric, Alex Groce, Chaoqiang Zhang, Rohan Sharma, Amin Alipour, and Darko Marinov. Comparing non-adequate test suites using coverage criteria. In *International Symposium on Software Testing and Analysis*, pages 302–313, 2013.
- [12] Muhammad Ali Gulzar, Yongkang Zhu, and Xiaofeng Han. Perception and practices of differential testing. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 71–80, 2019.
- [13] John McGehee. pyfakefs implements a fake file system that mocks the python file system modules. <https://github.com/jmcgeheeiv/pyfakefs>.
- [14] Alex Groce and mrbean bremen. Pyfakefs harness. <https://github.com/agroce/tstl/blob/master/examples/pyfakefs/fakeosmodule.tstl>.
- [15] Marjan Mernik, Jan Heering, and Anthony M. Sloane. When and how to develop domain-specific languages. *ACM Computing Surveys*, 37(4):316–344, 2005.
- [16] Martin Fowler. *Domain-Specific Languages*. Addison-Wesley Professional, Upper Saddle River, NJ, 2010. With Rebecca Parsons.
- [17] Alex Groce and Jervis Pinto. A little language for testing. In *NASA Formal Methods Symposium*, pages 204–218, 2015.
- [18] Josie Holmes, Alex Groce, Jervis Pinto, Pranjal Mittal, Pooria Azimi, Kevin Kellar, and James O’Brien. TSTL: the template scripting testing language. *International Journal on Software Tools for Technology Transfer*, 20(1):57–78, February 2018.
- [19] Eric Gamma and Kent Beck. JUnit 5. <http://junit.org/junit5/>.
- [20] Google Test. <https://github.com/google/googletest>, 2008.
- [21] <http://www.cs.cmu.edu/~modelcheck/cbmc/>.
- [22] Google. Fuzztest. <https://github.com/google/fuzztest>, 2022.

- [23] Peter Goodman and Alex Groce. DeepState: Symbolic unit testing for C and C++. In *NDSS Workshop on Binary Analysis Research*, 2018.
- [24] Alastair Reid, Luke Church, Shaked Flur, Sarah de Haas, Maritza Johnson, and Ben Laurie. Towards making formal methods normal: meeting developers where they are, 2020.
- [25] Trail of Bits. DeepState: A unit test-like interface for fuzzing and symbolic execution. <https://github.com/trailofbits/deepstate>, 2018.
- [26] Alex Groce, Jervis Pinto, Pooria Azimi, and Pranjal Mittal. TSTL: a language and tool for testing (demo). In *ACM International Symposium on Software Testing and Analysis*, pages 414–417, 2015.
- [27] Alex Groce, Jervis Pinto, Pooria Azimi, Pranjal Mittal, Josie Holmes, and Kevin Kellar. TSTL: the template scripting testing language. <https://github.com/agroce/tstl>, May 2015.
- [28] George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. Evaluating fuzz testing. *arXiv preprint arXiv:1808.09700*, 2018.
- [29] Daniel Lehmann and Michael Pradel. Feedback-directed differential testing of interactive debuggers. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 610–620, 2018.
- [30] Alex Groce, Iftekhar Ahmed, Josselin Feist, Gustavo Grieco, Jiri Gesi, Mehran Meidani, and Qihong Chen. Evaluating and improving static analysis tools via differential mutation analysis. In *21st IEEE International Conference on Software Quality, Reliability and Security, QRS 2021, Hainan, China, December 6-10, 2021*, pages 207–218. IEEE, 2021.
- [31] Gergő Barany. Finding missed compiler optimizations by differential testing. In *Proceedings of the 27th international conference on compiler construction*, pages 82–92, 2018.
- [32] Yuting Chen, Ting Su, and Zhendong Su. Deep differential testing of jvm implementations. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pages 1257–1268. IEEE, 2019.
- [33] Yuting Chen, Ting Su, Chengnian Sun, Zhendong Su, and Jianjun Zhao. Coverage-directed differential testing of jvm implementations. In *proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 85–99, 2016.
- [34] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. Finding and understanding bugs in C compilers. In *ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 283–294, 2011.
- [35] Steffen Herbold and Steffen Tunkel. Differential testing for machine learning: an analysis for classification algorithms beyond deep learning. *Empirical Software Engineering*, 28(2):34, 2023.
- [36] Meiziniu Li, Dongze Li, Jianmeng Liu, Jialun Cao, Yongqiang Tian, and Shing-Chi Cheung. Enhancing differential testing with llms for testing deep learning libraries. *ACM Transactions on Software Engineering and Methodology*, 35(4):1–39, 2026.
- [37] Jaeyeon Jung, Anmol Sheth, Ben Greenstein, David Wetherall, Gabriel Maganis, and Tadayoshi Kohno. Privacy oracle: a system for finding application leaks with black box differential testing. In *Proceedings of the 15th ACM conference on Computer and communications security*, pages 279–288, 2008.
- [38] Yuting Chen and Zhendong Su. Guided differential testing of certificate validation in ssl/tls implementations. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, pages 793–804, 2015.
- [39] Pengbo Nie, Chengcheng Wan, Jiayu Zhu, Ziyi Lin, Yuting Chen, and Zhendong Su. Coverage-directed differential testing of x. 509 certificate validation in ssl/tls implementations. *ACM Transactions on Software Engineering and Methodology*, 32(1):1–32, 2023.
- [40] Mark Guzdial. A media computation course for non-majors. In *Proceedings of the 8th Annual Conference on Innovation and Technology in Computer Science Education, ITiCSE '03*, pages 104–108, New York, NY, USA, 2003. ACM.
- [41] Alex Groce, Goutamkumar Tulajappa Kalburgi, Claire Le Goues, Kush Jain, and Rahul Gopinath. First, fuzz the mutants. In *1st International Fuzzing Workshop*, 2022.

- [42] Kush Jain, Goutamkumar Tulajappa Kalburgi, Claire Le Goues, and Alex Groce. Mind the gap: The difference between coverage and mutation score can guide testing efforts. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, pages 102–113, 2023.
- [43] Kush Jain, Uri Alon, Alex Groce, and Claire Le Goues. Contextual predictive mutation testing. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2023, page 250–261, New York, NY, USA, 2023. Association for Computing Machinery.
- [44] Sourav Deb, Kush Jain, Rijnard van Tonder, Claire Le Goues, and Alex Groce. Syntax is all you need: A universal-language approach to mutant generation. In *ACM International Conference on the Foundations of Software Engineering*, 2024. Accepted for publication.